

Anexo 1 — default, null y Empty en C# (Parte 1 de 2)

Curso: Arquitectura de Software Profesional con Clean Architecture, DDD y SOLID
Complemento del **Módulo 1 — Entidades** · Prerrequisito del **Módulo 2 — Value Objects**

Objetivo del anexo

Comprender con precisión qué representan `default`, `null` y `string.Empty` en C#, cómo se comportan según el tipo de dato, qué excepciones pueden provocar y cómo detectarlos correctamente. Este conocimiento es fundamental para escribir validaciones robustas en Value Objects y entidades del dominio.

`default` → valor inicial automático de un tipo cuando no se asigna nada
`null` → ausencia intencional de referencia: el objeto no existe
`Empty` → valor presente pero vacío (aplica principalmente a `string`)

Contenido de este anexo

Tema 1	¿Qué es <code>default</code> ?
Tema 2	¿Qué es <code>null</code> ?
Tema 3	¿Qué es <code>string.Empty</code> ?
Tema 4	Tabla comparativa por tipo de dato
Tema 5	Excepciones que pueden provocar
Tema 6	Cómo detectar cada uno correctamente
Tema 7 — Parte 2	Errores comunes
Tema 8 — Parte 2	Qué NO hacer
Tema 9 — Parte 2	Ejemplo práctico integrado
Tema 10 — Parte 2	Actividades prácticas

Tema 1 — ¿Qué es `default`?

1. ¿Qué es?

La palabra clave `default` en C# representa el **valor inicial que tiene un tipo cuando no se le asigna ningún valor explícito**. No es un error ni una ausencia intencional: es el punto de partida automático de cualquier variable, determinado por el compilador según el tipo declarado.

2. Analogía

Es como un formulario recién impreso: todos los campos existen, pero ninguno fue completado. El campo "nombre" tiene un espacio en blanco, el campo "cantidad" tiene un cero, el campo "fecha" tiene la fecha mínima posible. El formulario *existe físicamente*, pero sus valores son los que vienen "de fábrica".

3. ¿Qué valor tiene default según el tipo?

<code>int</code>	<code>0</code>
<code>bool</code>	<code>false</code>
<code>decimal</code>	<code>0m</code>
<code>DateTime</code>	<code>01/01/0001 00:00:00</code> — el mínimo posible en .NET
<code>Guid</code>	<code>00000000-0000-0000-0000-000000000000</code> — todos ceros
<code>string</code>	<code>null</code> — string es una clase, los tipos referencia tienen default = null
Cualquier clase	<code>null</code> — todos los tipos por referencia tienen default = null

```

1 // — Tipos de valor (value types): default es un valor numérico o lógico —
2 int numero = default;      // 0
3 bool activo = default;     // false
4 decimal precio = default;  // 0m
5 DateTime fecha = default;  // 01/01/0001 00:00:00
6 Guid id = default;        // 00000000-0000-0000-0000-000000000000
7
8 // — Tipos por referencia (reference types): default es null —
9 string nombre = default;   // null — string es una clase, no un struct
10 Customer customer = default; // null — toda clase tiene default = null
11
12 // — Uso como centinela — patrón usado en BaseEntity del curso —
13 if (id == default)         // true si id es 00000000-0000-0000-0000-000000000000
14     id = Guid.NewGuid();   // es una entidad nueva → generar uno
  
```



```
15 else
16     // id tiene valor real → es materialización desde BD → conservarlo
```

🔍 ¿Por qué usamos default en BaseEntity?

- Si `id == default` → nadie pasó un `Guid` real → es una entidad nueva → generar `Guid.NewGuid()`.
- Si `id != default` → EF Core pasó el `Guid` guardado en BD → conservarlo.
- Este patrón permite que un único constructor cumpla los dos roles sin ambigüedad ni lógica adicional.

Tema 2 — ¿Qué es `null`?

1. ¿Qué es?

`null` representa la **ausencia intencional de referencia a un objeto**. Una variable que contiene `null` no apunta a ninguna dirección de memoria: el objeto literalmente *no existe*.

Solo los **tipos por referencia** (clases, interfaces, arrays, delegates) y los **tipos de valor nullable** (`int?`, `DateTime?`, etc.) pueden almacenar `null`. Los tipos de valor puros (`int`, `Guid`, `bool`) **no pueden ser null** sin el operador `?`.

2. Analogía

Una variable es como una hoja de papel que dice "buscar el objeto en la dirección X".

`null` es una hoja completamente en blanco: no indica ninguna dirección. Si el programa intenta seguir esa dirección para encontrar el objeto, lanza una excepción porque no hay adónde ir: esto se llama **NullReferenceException**.

3. Tipos que pueden ser null

```
1 // — Tipos por referencia — pueden ser null por naturaleza —————
2 string? nombre = null;           // string es una clase → puede ser null
3 Customer? customer = null;      // toda clase puede ser null
4
5 // — Tipos de valor nullable — se agrega ? para permitir null —————
6 int? edad = null;               // int? = "número que puede no existir aún"
7 DateTime? modificadoEn = null; // DateTime? = "fecha que puede no haber ocurrido"
8 Guid? usuarioId = null;        // Guid? = "id que puede no estar asignado"
9
10 // — Tipos de valor puros — NO pueden ser null sin ? —————
11 int numero = null;             // ✗ ERROR DE COMPILACIÓN
12 Guid id = null;               // ✗ ERROR DE COMPILACIÓN
13 DateTime fecha = null;        // ✗ ERROR DE COMPILACIÓN
```

4. ¿Por qué ModifiedOn es DateTime? en BaseEntity?

Usamos `DateTime? ModifiedOn` porque esta fecha puede no existir: si el registro nunca fue modificado, no hay fecha de modificación. `null` aquí significa "este evento *todavía no ocurrió*", lo cual es semánticamente correcto y evita usar valores centinela artificiales.



Comparación semántica de las tres opciones:

```
DateTime ModifiedOn = DateTime.MinValue → ¿Fue modificado el 01/01/0001? No tiene sentido.  
DateTime ModifiedOn = DateTime.UtcNow   → ¿Siempre fue modificado al crearse? Tampoco.  
DateTime? ModifiedOn = null              → ☒ "Aún no fue modificado." Claro y correcto.
```

Tema 3 — ¿Qué es `string.Empty`?

1. ¿Qué es?

`string.Empty` es una constante de C# que representa una **cadena de texto que existe pero no contiene ningún carácter**. Es exactamente equivalente a `""`, pero más explícita en comunicar la intención del desarrollador.

La diferencia con `null` es fundamental: `null` significa que el string *no existe*; `string.Empty` significa que el string *existe pero está vacío*.

2. Analogía

Imaginar tres sobres:

- **null**: el sobre no existe. No hay sobre. No hay nada de nada.
- **string.Empty / ""**: el sobre existe y está cerrado, pero adentro no hay nada.
- **" "** (solo espacios): el sobre existe, adentro hay una hoja, pero la hoja está en blanco.
- **"Juan"**: el sobre existe y adentro hay una hoja con contenido real útil.

3. Las cuatro formas de "string sin valor útil"

```
1 // Caso 1: null – la variable no apunta a ningún objeto string
2 string? nombre = null;
3 nombre.Length; // ❌ NullReferenceException: el objeto no existe
4
5 // Caso 2: string.Empty – existe, pero sin caracteres
6 string nombre = string.Empty; // equivalente a ""
7 nombre.Length; // ✅ devuelve 0 – el objeto existe, no lanza excepción
8
9 // Caso 3: whitespace – existe, tiene caracteres, pero son solo espacios
10 string nombre = " "; // tres espacios – parece vacío visualmente
11 nombre.Length; // devuelve 3 – ¡length no lo considera vacío!
12
13 // Caso 4: valor real útil
14 string nombre = "Juan"; // existe y tiene contenido útil
15 nombre.Length; // devuelve 4
```

4. ¿Por qué usamos `= string.Empty` en las propiedades de Customer?

En `Customer` declaramos las propiedades con `= string.Empty` como valor inicial:

```
1 public string FullName { get; private set; } = string.Empty;
```

Esto garantiza que la propiedad **nunca sea null** durante el ciclo de vida del objeto. El compilador de C# con *Nullable Reference Types* activado lo exige: si no asignamos un valor inicial no-null, advierte que la propiedad podría ser null sin querer.

`string.Empty` es la forma más explícita de decir "esta propiedad existe, está vacía, y el método `Create()` la llenará".

Tema 4 — Tabla comparativa por tipo de dato

Tipo	default	Puede ser null	Empty / vacío equivalente
<code>int</code>	<code>0</code>	✗ No	<code>0</code> (depende del contexto)
<code>int?</code>	<code>null</code>	✓ Sí	<code>0</code> o <code>null</code>
<code>bool</code>	<code>false</code>	✗ No	<code>false</code>
<code>decimal</code>	<code>0m</code>	✗ No	<code>0m</code>
<code>DateTime</code>	<code>01/01/0001 00:00:00</code>	✗ No	<code>DateTime.MinValue</code> = mismo que default
<code>DateTime?</code>	<code>null</code>	✓ Sí	<code>null</code> = "no ocurrió aún"
<code>Guid</code>	<code>00000000-...-0000</code>	✗ No	<code>Guid.Empty</code> = exactamente igual a default
<code>Guid?</code>	<code>null</code>	✓ Sí	<code>Guid.Empty</code> o <code>null</code>
<code>string</code>	<code>null</code>	✓ Sí (es clase)	<code>""</code> o <code>string.Empty</code>
Cualquier clase	<code>null</code>	✓ Sí	Depende de la clase

💡 `Guid.Empty` vs `default(Guid)` — ¿son lo mismo?

- `Guid.Empty` y `default(Guid)` son **exactamente el mismo valor**: `00000000-0000-0000-0000-000000000000`.
- En `BaseEntity` usamos `default` como centinela para detectar "no fue pasado un Guid real" porque es el valor de un `Guid` sin inicializar.
- Usamos `Guid.Empty` al asignar "ningún usuario" para comunicar intención explícita.
- En código de validación, comparar con cualquiera de los dos funciona: `id == default` es igual a `id == Guid.Empty`.

Tema 5 — Excepciones que pueden provocar

1. `NullPointerException` — la más común

Ocurre al intentar usar un objeto que es `null`: acceder a una propiedad, llamar a un método o indexar un array. Es un bug que el compilador moderno puede advertir con los *Nullable Reference Types*.

```
1  string? email = null;
2
3  // ❌ NullPointerException: email es null, no existe el objeto string
4  var lower = email.ToLower();
5
6  // ❌ NullPointerException: email es null, no tiene propiedad Length
7  var largo = email.Length;
8
9  // ✅ Verificar antes de usar — dos formas equivalentes
10 if (email is not null)
11     var lower = email.ToLower(); // seguro: compilador sabe que no es null aquí
```

2. `ArgumentNullException` — validación intencional con **Fail Fast**

Es la excepción que nosotros lanzamos explícitamente cuando un parámetro obligatorio llega como `null`. A diferencia de `NullPointerException` (que es un bug accidental), `ArgumentNullException` es una decisión de diseño que aplica el principio **Fail Fast**: fallar rápido con un mensaje claro.

```
1  // ✅ Fail Fast: detectar null lo antes posible con mensaje descriptivo
2  public static Email Create(string? value)
3  {
4      // primer control: null
5      if (value is null)
6          throw new ArgumentNullException(nameof(value), "El email no puede ser null.");
7
8      // segundo control: vacío o solo espacios
9      if (string.IsNullOrEmpty(value))
10         throw new ArgumentException("El email no puede estar vacío.", nameof(value));
11
12     // a partir de aquí el compilador confirma que value no es null ni vacío
13     // continuar con validaciones de formato...
14 }
```

3. `InvalidOperationException` al acceder a nullable sin verificar

```
1  DateTime? modificadoEn = null;
2
3  // ❌ InvalidOperationException: el nullable no tiene valor, .Value falla
4  DateTime fecha = modificadoEn.Value;
```



```
5
6 // ✅ Opción 1: verificar con HasValue antes de acceder
7 if (modificadoEn.HasValue)
8     DateTime fecha = modificadoEn.Value; // seguro
9
10 // ✅ Opción 2: operador ?? – dar un valor por defecto si es null
11 DateTime fecha = modificadoEn ?? DateTime.UtcNow;
12
13 // ✅ Opción 3: GetValueOrDefault – devuelve default(DateTime) si es null
14 DateTime fecha = modificadoEn.GetValueOrDefault();
```

Tema 6 — Cómo detectar cada uno correctamente

1. Detectar null

```
1 // — Forma clásica —————
2 if (valor == null) { /* es null */ }
3 if (valor != null) { /* no es null, seguro de usar */ }
4
5 // — Pattern matching – forma moderna y preferida (C# 7+) —————
6 if (valor is null) { /* es null */ }
7 if (valor is not null) { /* no es null */ }
8
9 // — Operador de propagación nula ?. – no lanza excepción si es null ———
10 var largo = valor?.Length; // devuelve null si valor es null, en lugar de lanzar excepción
11
12 // — Operador ?? – valor por defecto si es null —————
13 var nombre = valor ?? string.Empty; // si valor es null → usar string.Empty
```

2. Detectar string vacío, null o solo espacios

```
1 // — string.IsNullOrEmpty: detecta null y "" —————
2 // NO detecta " " (espacios): Length = 3, no es "vacío" para este método
3 string.IsNullOrEmpty(null); // true
4 string.IsNullOrEmpty(""); // true
5 string.IsNullOrEmpty(" "); // false ← ¡OJO! Solo espacios NO se detectan
6 string.IsNullOrEmpty("Juan"); // false
7
8 // — string.IsNullOrWhiteSpace: detecta null, "" y " " —————
9 // ✅ Este es el que usamos en validaciones del dominio – cubre todos los casos
10 string.IsNullOrWhiteSpace(null); // true
11 string.IsNullOrWhiteSpace(""); // true
12 string.IsNullOrWhiteSpace(" "); // true ← detecta correctamente
13 string.IsNullOrWhiteSpace("Juan"); // false – tiene contenido útil
```

3. Detectar default en tipos de valor

```
1 // — Guid: comparar con default o con Guid.Empty (son equivalentes) ———
2 Guid id = default;
3 if (id == default) { } // true – forma usada en BaseEntity
4 if (id == Guid.Empty) { } // true – equivalente, más explícito
5
6 // — DateTime: comparar con default o con DateTime.MinValue —————
7 DateTime fecha = default;
8 if (fecha == default) { } // true
9 if (fecha == DateTime.MinValue) { } // true – equivalente
10
11 // — DateTime? nullable: verificar con HasValue o comparar con null ———
12 DateTime? modificadoEn = null;
13 if (!modificadoEn.HasValue) { } // true – nunca fue modificado
14 if (modificadoEn is null) { } // true – equivalente, forma moderna
```

✦ Resumen de detección — regla rápida para el dominio

- Para **strings** en validaciones del dominio: usar siempre `string.IsNullOrEmpty()` — cubre null, vacío y solo espacios en una sola llamada.
- Para **Guid**: comparar con `default` o `Guid.Empty` — son equivalentes.
- Para **DateTime?** nullable: usar `.HasValue` o `is null` antes de acceder a `.Value`.
- Para **null** en general: preferir el pattern matching `is null` / `is not null` — es más claro y el compilador lo entiende mejor.

▶ Continúa en **Anexo 1B** — Errores comunes, Qué NO hacer, Ejemplo práctico integrado y Actividades.